Algoritmi paraleli si distribuiți

10. Algoritmi pe grafuri (II): Prim, Kruskal, APSP (n x Dijkstra, Roy-Floyd), componente conexe, SSSP pe grafuri rare.

+ Neo4j

Arbore parţial de cost minim (APM)
(en., MST = Minimum cost spanning tree)

- ***Arborele de acoperire*** pentru un graf nedirecționat G(V, E) este un subgraf al lui G care este un arbore care conține toate nodurile din G.
  - Subgraf = submultime de muchii din E
  - Arbore = graf (nedirecționat) fără cicluri
  - De acoperire = conține toate nodurile din G
- APM pentru un graf ponderat este arborele de acoperire cu cost minim.
  - Costul = suma costurilor muchiilor din APM
- Algoritmi care rezolvă aceasta problemă
  - **Prim**, Kruskal, Boruvka

Algoritmul lui Prim pentru APM

- Este greedy și iterativ
- Alege un nod de start oarecare
- Alege un nou nod (ca Dijkstra) care este inclus in APM
- Timp $O(n^2)$

Scopul paralelizării:

- Obținerea accelerării în urma rulării pe mai multe procesoare

- Procesarea de grafuri mari care nu pot fi rezolvate de un singur procesor

Algoritmul lui Prim: formulare paralelă

- Este dificil de efectuat iterații diferite din bucla principală deoarece d[v] se poate modifica de fiecare dată.

while VT != V **do** *//VT sunt nodurile din APM*

find a vertex **u** such that **d[u] := min{d[v] | v** ∈ **(V - VT)}**;

VT := VT ∪ {u}; *//u este nodul cucerit*

for all v ∈ (V - VT) **do**

d[v] := min{d[v], w(u, v)};

endwhile

- Totusi, fiecare iterație se poate excuta în paralel.

Algoritmul lui Prim: formulare paralelă (cont.)

- Împărțim nodurile în **p** submulțimi $\mathbf{V_i}$, unde ***i = 0, 1, ..., p-1***
- Fiecare proces Pi determină

$$d_i[u]=\min\{d_i[v] \mid v \in (V-VT) \cap V_i\}$$

- Optimul (i.e., minimul) global se obține prin reducere toti-la-unul (en., all-to-one reduction)
- Un nod nou care este adaugat in VT este de asemenea transmis către toate procesele
  - Astfel, noi valori pentru d[v] sunt calculate pentru nodurile locale
- Timpul total este $\mathbf{O(n^2/p) + O(n \log p)}$, (calcul + comunicare)

Algoritmul lui Prim: formulare paralelă (cont.)

Contextul problemei

- Reprezentarea grafului: matrice de adiacență
- Graful este distribuit catre procesoare
    - V este împărțită în submulțimi
    - Fiecare procesor se ocupă de o submulțime de noduri si toate muchiile acestora.

Algoritm paralel referință

- Nodul 0 este adăugat în APM (i.e., soluție)
- Fiecare procesor **pi**
    - Determină cel mai apropiat nod de APM-ul curent
        - La primul pas APM-ul este alcătuit doar din nodul 0
    - Transmite muchia de cost minim către procesul părinte
- Reducerea MIN (executată in procesul părinte)
    - Procesul părinte determină minimul global. Acest pas are ca rezultat determinarea/cucerirea uni nod care este adaugat la APM.
    - Nodul nou adăugat/cucerit este transmis (en. , broadcast) către toate prosoarele.
        - Procesoarele fac update in partea de graf gestionată.
- Acești pași se repetă până când APM-ul contine toate nodurile din V.

Algoritmul lui Prim: formulare paralelă (cont.)

- Limitări ale variantei paralele de referință
  - Pe grafurile mici (<10K de noduri) accelerarea este minimală
  - Pe grafuri mari (25K-100K de noduri) costurile suplimetare (en., overhead) generate de comunicare domină timpul necesar procesărilor ceea ce conduce la lipsa accelerării.
- Posibile îmbunătățiri ale performanței
  - Utilizarea mobilelor binome sau Fibonacci pentru determinarea minimului
  - Utilizarea listei de adiacență pentru reprezentarea grafului

Algoritmul lui Kruskal

- Algoritmul lui Kruskal construiește o multime de APM în paralel.
- Inițial, fiecare nod este un arbore separat.
- Se sortează muchiile în ordine crescătoare a costurilor.
- Muchiile sunt analizate in ordinea costurilor
  - Dacă muchia conectează doi arbori distincți atunci acestia sunt uniti
  - Dacă muchia uneste două noduri ale aceluiași arbore atunci ea face un ciclu și deci este ignorată.
- Algoritmul se termină când
  - Nu mai avem muchii
  - Avem un singur arbore, care este APM-ul căutat.
- Obs: Implementarea clasică este cu Union-Find.

Algoritmul lui Kruskal: formulare paralelă

- **Varianta 1. Prin paralelizarea sortării**

Având in vedere că algoritmul are două faze (i.e., Sortare si Parcurgere) o primă opțiune este paralelizarea operației de sortare urmată de parcurgere în mod asemănător ca în varianta secvențială.

Algoritmul lui Kruskal: formulare paralelă (cont.)

Varianta 2. Prin distribuția grafului pe procesoare

- Se partiționează graful în mod asemănător ca la algoritmul lui Prim
- Fiecare process pi determină APM-ul local, $\mathbf{F_i}$ corespunzător muchiilor din partiția $\mathbf{V_i}$ prin aplicarea algortmului lui Kruskal.
- Procesele unesc APM-urile locale intr-unul global.
  - Fie a si b doua procese care și unesc APM-urile determinate, $\mathbf{F_a}$ si $\mathbf{F_b}$
  - Procesul a trimite pe F_a cître procesul b, care determină un nou APM din $F_a \cup F_b$
  - Unirea APM-urilor continuă până când determinăm AMP-ul final.

Algoritmul lui Kruskal: formulare paralelă (cont.)

- Unirea APM-urilor $\mathbf{F_a}$ si $\mathbf{F_b}$ se poate realiza prin rularea lui Kruskal pe $\mathbf{F_a} \cup \mathbf{F_b}$
- Detrminarea unui APM local costă $O(n^2/p)$
- Avem in total *log p* uniri, fiecare costă $O(n^2 \log p)$
- În timpul unei uniri procesul transmite maxim O(n) muchii

$$\mathbf{T_p = O(n^2/p) + O(n^2 \log p)}$$

Algoritmul paralel este eficient pentru ***p = O(n/logn)***

Algoritmul lui Dijkstra: formulare paralelă

- Utilizează aceeași abordare ca algoritmul lui Prim pentru APM.
  - Pas 1. Matricea de adiacență este partiționată pe coloane (i.e., subgraf cu o submulțime de noduri si tote muchiile acestora)
  - Pas 2. Fiecare proces selectează local nodul cel mai apropiat de nodul sursă. Procesul părinte analizează toate soluțiile/propunerile locale și decide nodul cucerit.
  - Pas 3. Nodul cucerit este transmis (i.e., broadcast) către toate procesoarele pentru ca vectorul de distanșe să fie actualizat.
- Ca și în algoritmul lui Dijkstra secvențial, algoritmul se termină atunci când toate nodurile au fost cucerite (i.e., li s-a calculat costul minim)
  - Presupunem că graful constă din noduri la care se poate ajunge din nodul sursă.

Algoritmul lui Dijkstra pentru APSP (en., All-Pairs Shortest Paths)

- Variantă secvențială
  - Rezolvăm problema APSP prin **n** execuții ale algoritmului lui Dijkstra, cate una din fiecare nod al grafului.
  - Limitări:
    - Algoritmul nu funcționează corect pentru costuri negative.
    - Complexitatea este $O(n^3 \log n)$ pentru grafuri dense.
    - Pentru costuri negative (i.e., trebuie Bellman-Ford) complexitatea devine $O(n^4)$
- Variantă paralelă
  - Se partiționează sursele: fiecare apel de SSSP (Dijkstra/Bellman-Ford) se execută pe câte un procesor separat.
  - Se paralelizează SSSP pentru cresterea concurenței.

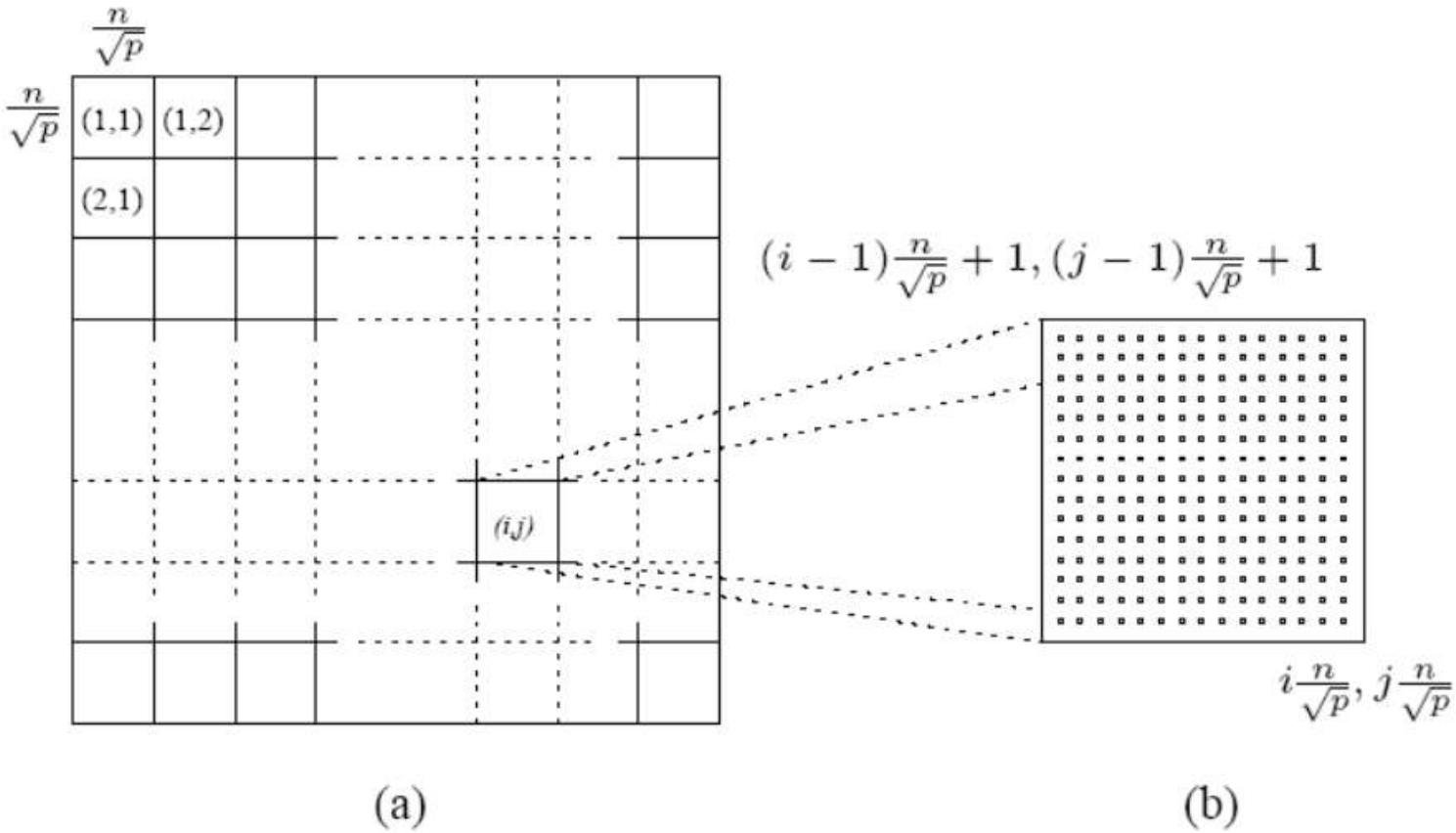
APSP cu Dijkstra: paralelizare cu partiționarea surselor

- Fiecare procesor $\mathbf{P_i}$ determină un SSSP de la nodul $\mathbf{v_i}$ la toate celelalte noduri prin executarea algoritmului lui Dijkstra secvențial.
  - \+ Nu apar costuri suplimentare cu comunicația, daca graful este replicat pe toate procesoarele.
  - \- Putem folosi maxim n procesoare.
  - $Tp = O(n^2)$ $O(n^2)$, ca il cazul algoritmului lui Prim.
  - \+ Paralelizarea SSSP face posibilă utilizarea a n^2 procesoare.

APSP: Algoritmul Roy-Floyd: varianta paralelă

- Se partiționeaza matricea de adiacență în p blocuri de dimensiune $(n/\sqrt{p})$ x $(n/\sqrt{p})$
- Fiecare procesor actualizează partea de matrice pe care o gestionează
- Pentru a determina $d_{l,r}^{(k-1)}$ fiecare procesor $\mathbf{P_{i,j}}$ trebuie să obțină valorile $d_{l,k}^{(k-1)}$ pentru toate valorile k ≠ r, și $d_{k,r}^{(k-1)}$ pentru toți k ≠ l.
- În timpul iterației k fiecare dintre cele $\sqrt{p}$ procese care conțin părți din rândul k transmit distanțele actualizate către $\sqrt{p} - 1$ procese din același coloană.
- Similar, fiecare dintre cele $\sqrt{p}$ procese care conțin părți din coloana k corespunzătoare transmit distanțele actualizate către $\sqrt{p} - 1$ procese din același rând.

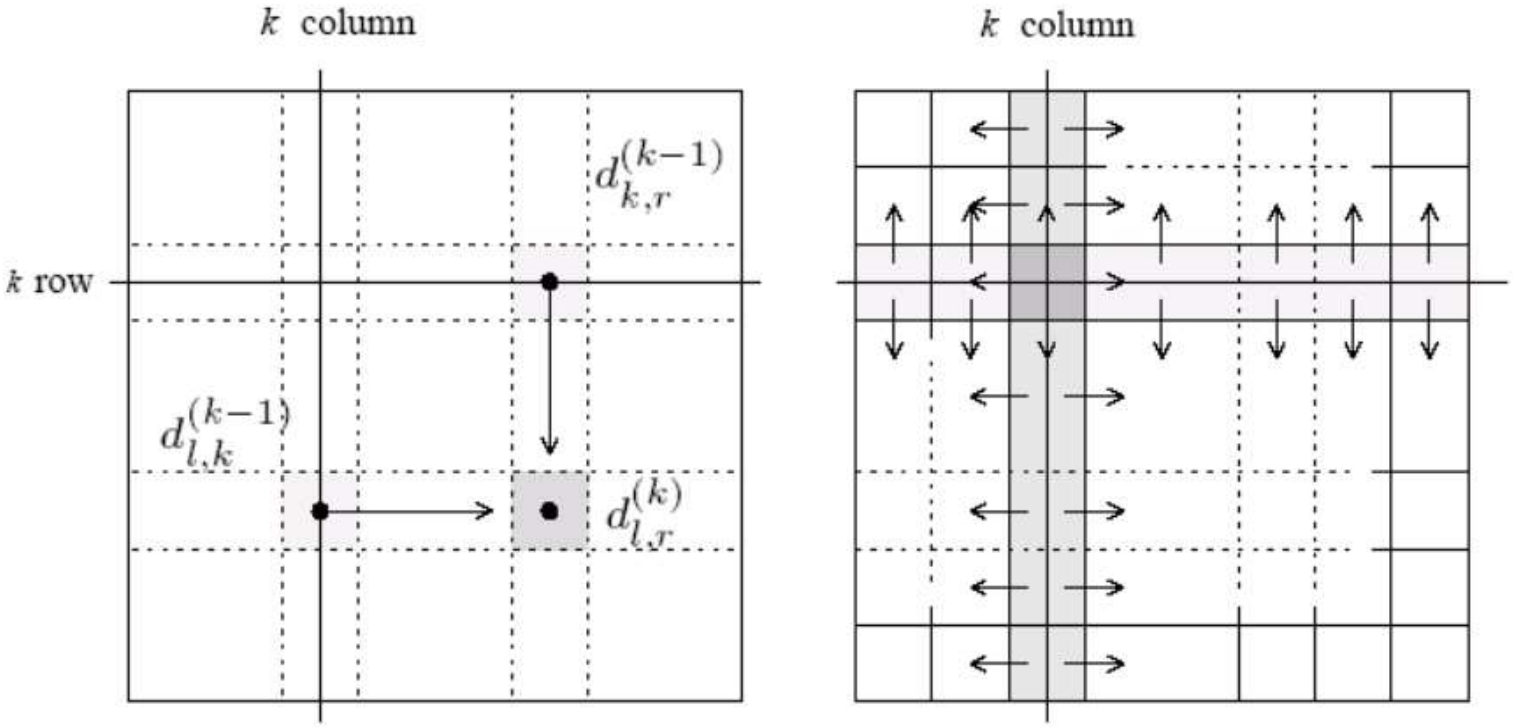
APSP: Algoritmul Roy-Floyd: varianta paralelă



a) Matricea de adiacență D(k) este împărțită în blocuri de dimensiune $\sqrt{p}$ x $\sqrt{p}$

b) Fiecare bloc este asignat unui procesor.

APSP: Algoritmul Roy-Floyd: varianta paralelă



Transmiterea distanțelor actualizate

APSP: Algoritmul Roy-Floyd: varianta paralelă

Procedure Roy_Floyd_2DBlock($D^{(0)}$)

for k=1 to n do

fiecare proces $P_{i,j}$ care are un segment al rândului k din $D^{(k-1)}$

- transmite distanțele către procesele $P_{*,j}$

fiecare proces $P_{i,j}$ care are un segment al coloanei k din $D^{(k-1)}$

- transmite distanțele către procesele $P_{i,*}$

fiecare proces așteaptă să primească segmentele necesare

fiecare proces $P_{i,j}$ calculează distanțele din blocul $D^{(k)}$ al matricii.

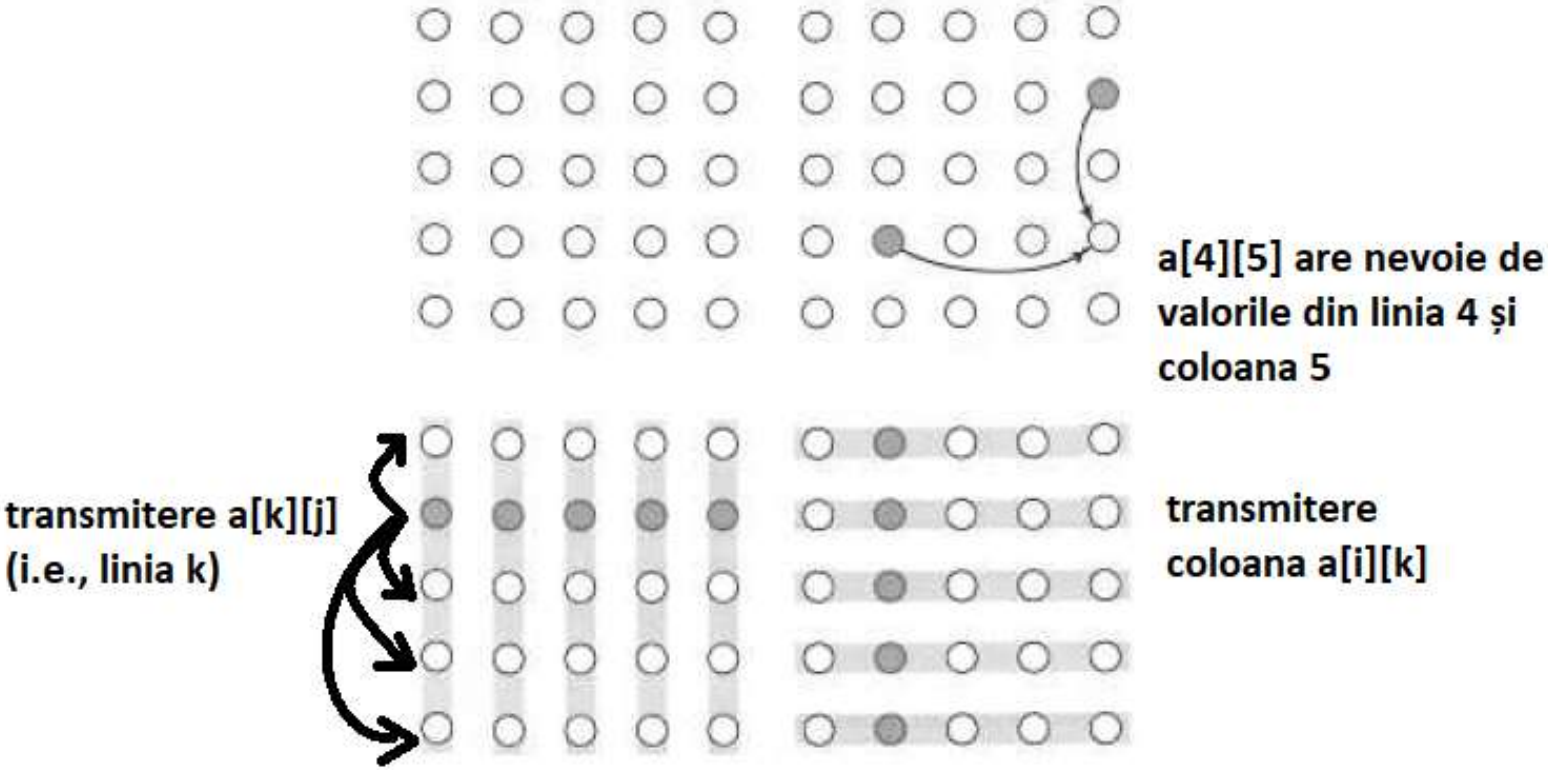
APSP: Algoritmul Roy-Floyd: varianta paralelă

- Putem actualiza valorile din matrice concurent?

a[i][j] = MIN (a[i][j], a[i][k]+a[k][j]);

- Da, deoarece rândul și coloana k nu se schimbă (i.e., fac o singură actualizare) în timpul iterației k.
- Din perspectiva metodologiei Foster
  - Partiționarea: fiecare a[i][j] este un task
  - Comunicarea: în timpul iterației k, actualizarea valorii din a[i][j] necesită valorile a[i][k] si a[k][j]
    - Se transmite (en., broadcast) a[i][k] către a[i][0], a[i][1], ..., a[i][n-1]
    - Se transmite a[k][j] către a[0][j], a[1][j], ..., a[n-1][j]

APSP: Algoritmul Roy-Floyd: varianta paralelă



APSP: Algoritmul Roy-Floyd: varianta paralelă

Alglomerarea si alocarea

- Un process este responsabil pentru o parte din matricea a[][]
  - Matricea a[][] este convenabil împărțită
- Părțile a[i][j] trebuie să fie de dimensiune egală
- O împărțire convenabilă este pe rânduri
  - Unui proces ii vor fi alocate un număr de rânduri condecutive din a[][]
  - Această împărțire este convenabilă deoarece valoarea a[i][k] se află la dispoziția aceluiași proces ca și a[i][j]
  - Totuși, a[k][j] se află în memoria altui proces => Este nevoie de comunicare!
    - Înainte de iterația k, procesul care detine rândul k il va transmite către toate celelalte procese.

APSP: Algoritmul Roy-Floyd: varianta paralelă

- La fiecare iterație a algoritmului rândul k și coloana k de procesoare realizează o transmisiune unu-la-toti (i.e.,one-to-all broadcast) către celelalte rânduri/coloane.
- Sunt transmise $n/\sqrt{p}$ elemente în O((n logp)/$\sqrt{p}$) timp.
- Sincronizarea durează O(log p).
- Timpul de calcul este O(n^2/p)
- Timpul de calcul paralel este

$$T_p = O\left(\frac{n^3}{p}\right) + O(\frac{n^2}{\sqrt{p}}\log p)$$

[calcul] + [comunicație]

APSP: Algoritmul Roy-Floyd: varianta paralelă

- Sincronizarea poate fi eliminată fără a afecta corectitudinea algoritmului
  - Un proces poate executa iterația k imediat ce a terminat iterația (k-1) iar părțile relevante din matricea $D^{(k-1)}$ sunt disponibile.
- Posibilități de îmbunătățire
  - Prin pipelining timpul paralel devine

$T_p = O\left(\frac{n^3}{p}\right) + O(n)$ si utilizarea a n^2 procese.

[calcul] + [comunicație]

Componente conexe: abordare paralelă

- Algoritmii secvențiali: DFS, BFS, Kosaraju.
- Abordare paralelă:
  - Distribuim graful între procesoarele disponibile
  - Fiecare procesor determină componentele conexe pentru subgraful primit
    - La acest moment avem p păduri de componente conexe
  - Pădurile de componente conexe sunt unite până când rămânem cu o singură pădure.
- Unirea eficientă a perechilor de componente conexe din păduri diferite trebuie să utilizeze Union-Find.
  - Union(x, y): unește două componente conexe care sunt disjuncte.
  - Find(x): returnază un pointer la reprezentantul componentei conexe care conține pe x. Fiecare componentă conexă are un reprezentat unic și diferit de celelalte componente din aceeași pădure.

Componente conexe: abordare paralelă (cont.)

- Unirea pădurilor A și B
  - Pentru fiecare muchie *(u, v)* din A de vor rula operațiile ***find(u)*** și ***find(v)*** pentru a determina dacă nodurile *u* și *v* sunt într-o componentă (i.e., au același reprezentatnt) ca un arbore din B.
    - Dacă cele două noduri sunt în doi arbori (i.e., componente conexe) diferiți atunci aceștia sunt uniți prin apelarea operației ***unite(u, v)***.
  - Unirea a două păduri A și B necesită cel mult *2(n-1)* operații ***find*** și *(n-1)* operații ***union***.

Componente conexe: abordare paralelă (cont.)

- Paralelizare prin alocarea de blocuri 1D
  - Matricea de adiacență n x n este împărțită în ***p*** blocuri.
  - Fiecare procesor va determina o padure de arbori de acoperire în $O(n^2/p)$
  - Unirea este realizată prin crearea unui arbore
    - Avem log p stagii în care realizăm unirile, și siecare stagiu durează O(n)
    - Costul total cu unirea este O(n log p)
  - În timpul fiecărei uniri, arbori de acoperire sunt trimiși între cei mai apropiați vecini.
    - Acest lucru crește costurie suplimentare cu comunicarea deoarece O(n) muchii trebuie transmise.

$$T_p = O\left(\frac{n^2}{p}\right) + O(n * \log p)$$

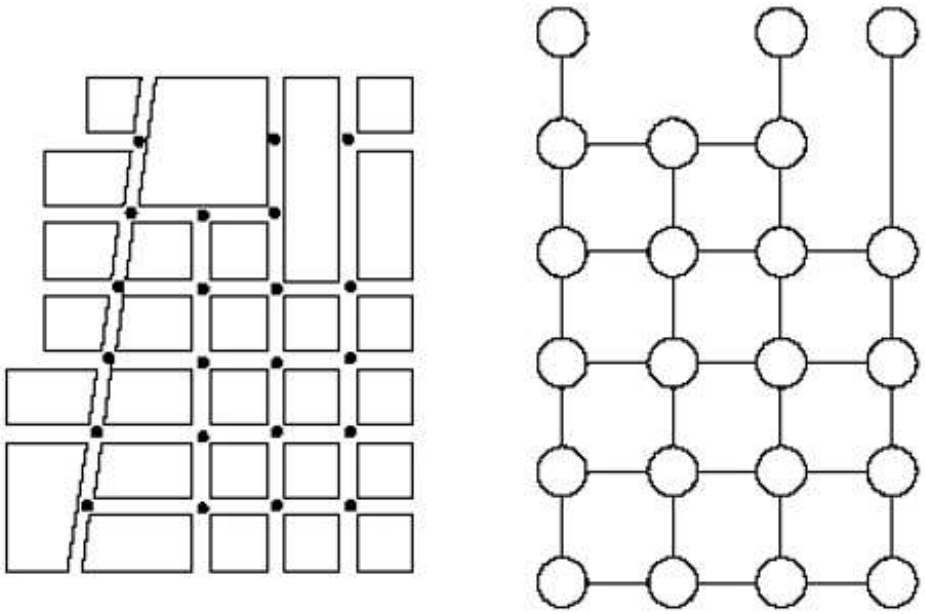
[calcule locale] + [unirea pădurilor]

Algoritmi pe grafuri rare

- Un graf G(V, E) este rar dacă $m << n^2$ (numarul de muchii este mult mai mic decat numarul maxim de muchii).
  - $n < m < n^2$, sau numarul de muchii este putin mai mare decât numarul de noduri.
  - Exemple de graduri rare: liniare (i.e., fiecare nod are doar doi vecini), grid (fiecare nod are 4 vecini) sau graf rar aleator.
- Dacă graful este rar atunci eficienta algoritmilor se poate îmbunătății
  - APM cu Prim: de la $O(n^2)$ la $O(m \log n)$.
- Pentru grafurile rare se preferă reprezentarea cu liste de adicacență.
- Partiționarea unei liste de adicacență poate fi dificilă
  - Realizăm balansarea în funcție de numărul de noduri?
  - Putem utiliza informatiile referitoare la gradul nodurilor?

Algoritmi pe grafuri rare: Exemplu

- Reprezentarea unei hărți a străzilor cu ajutorul grafurilor



Algoritmul lui Johnson: SSSP in grafuri rare

- Este echivalentul algoritmului lui Dijkstra pentru grafuri rare
- Algoritmul lui Dijkstra analizează numai nodurile adiacente celor cucerite.
- Algoritmul lui Johnson folosește o coadă de priorități pentru a gestiona valoarea ***l[v]*** pentru fiecare nod din $V-V_T$
- Observație: Menținerea unei ordin stricte conduce la restrangerea oportunităților de paralelizare.
- Soluție: Permiterea explorării mai multor noduri în paralel.
  - Se extrag simultan p noduri din coada de priorități
  - Se actualizează costurile vecinilor cu costurile minime.
  - În situația unei erori aceasta va fi descoperită ca o cale mai scurtă și nodul se va reintroduce cu costul minim actualizat.

Algoritmul lui Johnson: SSSP in grafuri rare (cont.)

- Dacă extragem și procesăm mai multe noduri din coadă atunci aceasta va determina un blocaj (en., *bottleneck*)

Soluția:

- Utilizarea mai multor cozi, câte una pentru fiecrae procesor.
  - Fiecare procesor își construiește propria sa coadă numai pentru nodurile pe care le gestionează.
- Atunci când procesul ***Pi*** extrage nodul ***u*** din ***Vi*** el trimite un mesaj proceselor care gestionează noduri adiacente cu ***u***.
- Atunci când **Pj** primește mesaj el actualizează valoarea lui **l[v]** în coada lui la valoarea **min{l[v],l[u] + w(u,v)}**
- Dacă un drum mai scurt este obținut către v de **Pj** atunci acesta este reactualizat în coada locală lui **Pi**.
- Algoritmul se termină atunci când toate cozile sunt goale.

Obs: Mai multe opțiuni de partiționare pot fi folosite pentru a exploata structura grafului care trebuie analizat.

Frameworks/biblioteci pentru procesarea grafurilor

Software	Limbaj/ interfață	paralelizare
SNAP (https://snap.stanford.edu/snap/)	C	Partajata (openMP)
Parallel Boost Graph Library (https://www.boost.org/doc/libs/1_63_0/libs/graph_parallel/doc/html/index.html)	C++	În memorie
Knowledge Discovery Toolbox (KDT) http://kdt.sourceforge.net/wiki/index.php/Main_Page	Python	În memorie
Pegasus https://www.cs.cmu.edu/~pegasus/index.htm	Hadoop	Pe disc.

Scalable R-MAT generator
https://www.boost.org/doc/libs/1_53_0/libs/graph_parallel/doc/html/scalable_rmat_generator.html
SNAP R-MAT generator
https://snap.stanford.edu/snappy/doc/reference/GenRMat.html

Baze de date graf

Ce sunt bazele de date graf?

- Muchie = relație între instanțe/înregistrări
- Nod = instanță/înregistrare

Caracteristici:

- Mai putin rigid structurate ca bazele de date relaționale (sql)
- Un nod (instanță/înregistare) poate foarte multe muchii (i.e., grad mare)
- O muchie poate avea mai multe caracteristici
  - Nu există nici o limitare în ceea ce privește gradul unui nod sau numarul de caracteristici
- Nodurile sunt definite de caracteristici, numite proprietăți (i.e., nume, ID, etc.)
- Nodurile se pot conecta prin muchii direcționate sau nedirecționate

Baze de date graf (cont.)

- Prin proiectare, bazele de date graf pot fi utilizate pentru reprezentarea rețelelor sociale
  - Nod = persoana, cu caracteristicile nume, vârstă, etc..
  - Muchie = relatia prieten, cu caracteristici de tipul pondere sau tip (i.e., coleg de servici, rudă, etc.)
- Aplicații utile
  - Structura grafului, sisteme de recomandare, etc.
- Caracteristici
  - Creare și întreținere, ca la orice baza de date.
  - Operații CRUD (i.e., create, read, update, and delete)
  - Interogări simplificate comparativ cu cele relationale

https://www.g2.com/categories/graph-databases

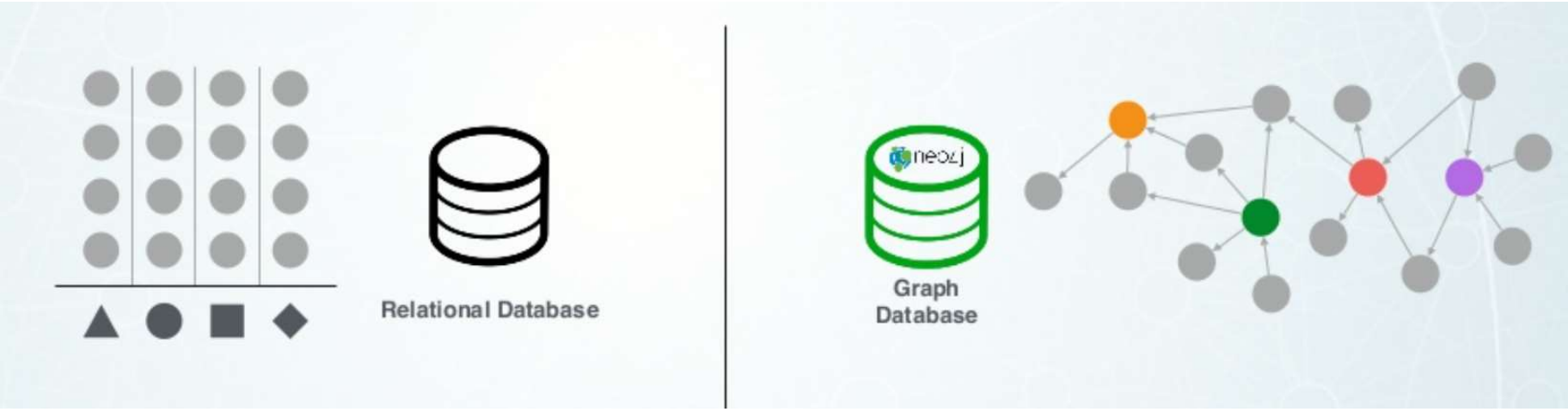
Neo4j (https://neo4j.com/)

- Conexiunile/legăturile dintre inregistrări sunt cel puțin la fel de importante ca și datele
- Google: paginile sunt nodurile, hiperlink-urile sunt muchii
- Facebook/Linkedin: utilizatorii sunt nodurile, relatiile sunt muchiile
- Amazon: utilizatorii/produsele sunt noduri, tranzacțiile sunt muchiile
- Limitele BD relaționale
  - O interogare poate lua in calcul multe date=> interogarile în timp real trebuie să aibă timp de răspuns mic
  - Interogarile pot analiza datele in mod nesecvențial, pe o cale aleatoare în date.

Neo4j (cont.)

- Neo4j
  - Gestionează și interoghează date care au structură de graf
  - Asigura parcurgere în timp real
  - Permite adăugarea facilă de date
  - Interogarile nu folosesc fisiere de index, și permit 1M+ hops/second cu ajutorul pointerilor
- Platforme
  - Linux, Solaris, Windows, macOS
  - Cloud: Amazon web services (AWS), Microsoft Azure, docker, etc.

Baze de date relaționale vs. Baze de date graf



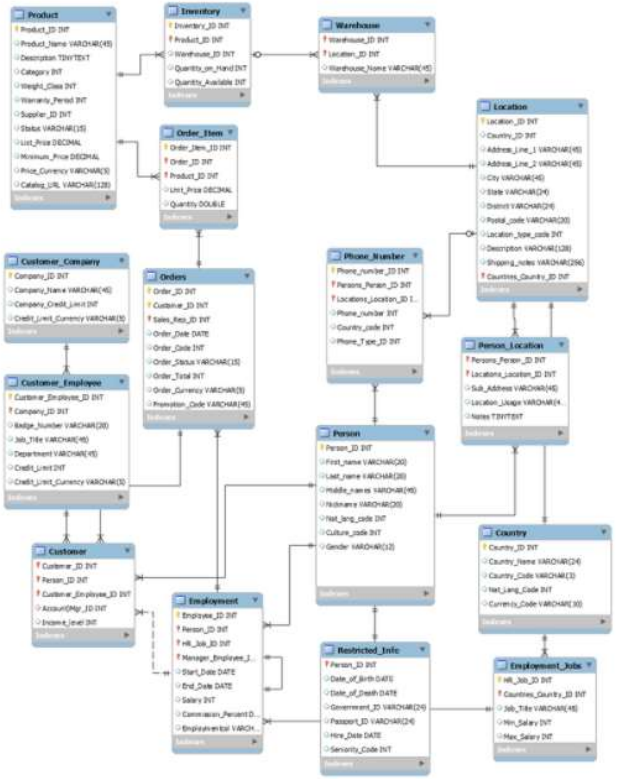
Folosim RDBMS atunci când avem:

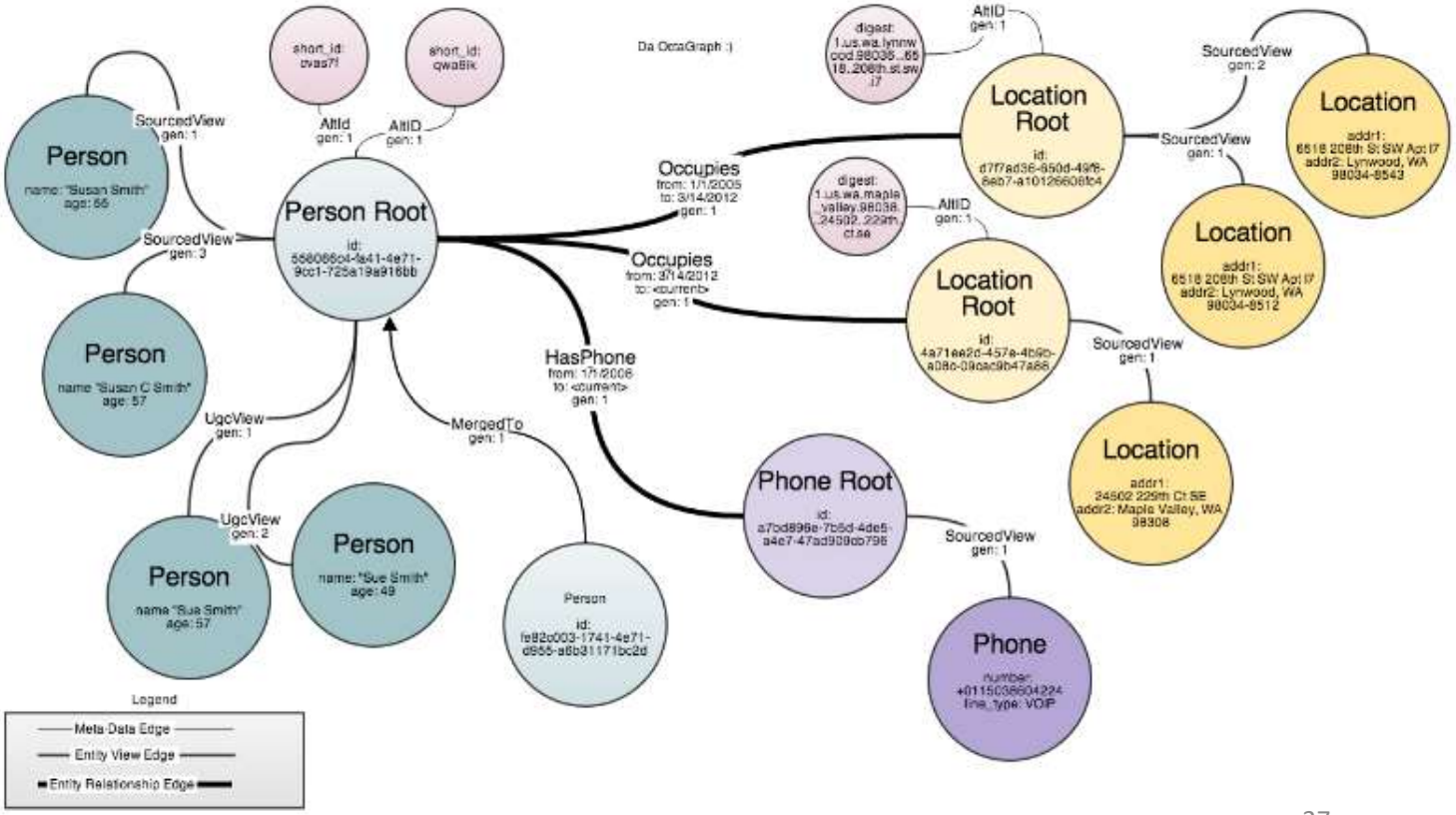
- Structură bine cunoscută și detrminată care nu se schimbă frecvent

Folosim BD graf atunci când avem:

- Structură dinamică în care topologia datelor este greu de evaluat.
- Relațiile dintre date (instanțe) au înțeles și valoare

De la RDBMS la BD Graf (dezvoltare/administrare)





Componentele unei baze de date graf

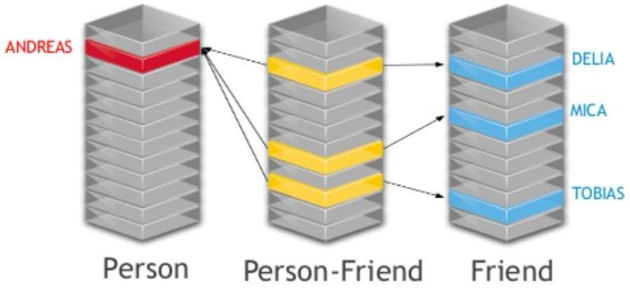
- Nodurile
  - Sunt obiecte în graf
  - Pot fi descrise de proprietăți de tipul nume-valoare
  - Pot avea etichete

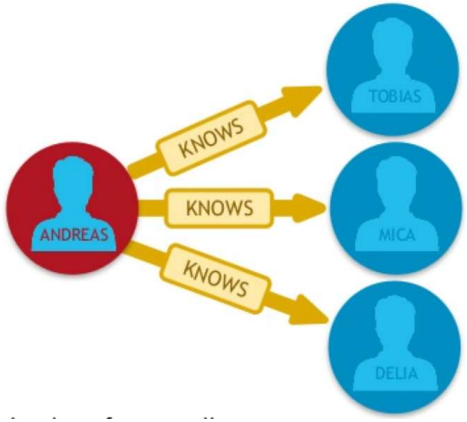
- Relațiile
  - Leagă nodurile în funcție de tip și direcție
  - Pot fi descrise de proprietăți de tipul nume-valoare

Avantaje ale utilizării BD graf

- Intuitivitate
- Viteză
- Interogările sql din bazele de date relationale devin ***cypher***.

Modelul relațional vs. Modelul graf





Fără indexare!

Cypher (i.e., sql in bazele de date relationale)

- Crearea datelor (i.e, noduri și muchii)

CREATE(:Person{name:”Ann”}) – [:LOVES]-> (:Person{name:”Ann”})

  - Noduri :Person{name:”Ann”}, where Person is label and name:”Ann” este o proprietate.
  - LOVES: este eticheta relației (i.e., atributul muchiei care leagă)

- Reprezentarea/crearea muchiilor bidirecționate/nedirecționate

MATCH(:Person{name:”Ann”}) – [:FB_FRIENDS]-**>** (:Person{name:”Dan”})

MATCH(:Person{name:”Ann”}) – [:FB_FRIENDS]- (:Person{name:”Dan”})

Cypher (cont)

- Cum ar arăta interogarea următoare cypher in sql?

```
MATCH (boss)-[:MANAGES*0..3]->(sub),
      (sub)-[:MANAGES*1..3]->(report)
WHERE boss.name = "John Doe"
RETURN sub.name AS Subordinate,
  count(report) AS Total
```

http://www.opencypher.org/

- GQL (graph query language) este omologul sql pentru BD graf.
- Utilizarea urmează aceiași pași ca la BD relaționale
  - Se crează modelul
  - Se populează cu date
  - Se interoghează
- Limbaje de programare
  - .Net, JS, Java, Python, rubby, php
- Jetbrains IDE (https://www.jetbrains.com/products/) vine cu Plugin pentru Graph Database Developers

https://neo4j.com/blog/jetbrains-ide-plugin-graph-database/

- Arhitectural
  - MVC cu inlocuirea BD (relationale, NoSQL, Hadoop) cu BD graf.

Algoritmi paraleli in Neo4j

PageRank

https://neo4j.com/docs/graph-data-science/current/algorithms/page-rank/

SSSP cu $\Delta-stepping$

https://neo4j.com/docs/graph-data-science/current/alpha-algorithms/single-source-shortest-path/

APM

https://neo4j.com/docs/graph-data-science/current/alpha-algorithms/minimum-weight-spanning-tree/

Machine Learning

https://neo4j.com/docs/graph-data-science/current/algorithms/node2vec/

https://neo4j.com/docs/graph-data-science/current/algorithms/ml-models/

Referințe:

Lončar, V., Škrbić, S., & Balaž, A. (2014). Parallelization of minimum spanning tree algorithms using distributed memory architectures. In Transactions on Engineering Technologies (pp. 543-554). Springer, Dordrecht. http://www.scl.rs/papers/Loncar-TET-Springer.pdf

Katsigiannis, A., Anastopoulos, N., Nikas, K., & Koziris, N. (2012, May). An approach to parallelize Kruskal's algorithm using helper threads. In 2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (pp. 1601-1610). IEEE.

http://www.cslab.ntua.gr/~anastop/files/papers/mtaap12kruskal.pdf